

CHƯƠNG TRÌNH THỰC TẬP HÈ UET 2026

BÁO CÁO TỔNG KẾT THỰC TẬP

Phát triển ECU chẩn đoán trên STM32

Giao tiếp CAN, ISO-TP và UDS

NHÓM 1

Nguyễn Đình Cường

Nguyễn Trường Giang

Bùi Tiến Dũng

Bùi Xuân Thanh

Đơn vị thực tập: FPT Software

Hà Nội, năm 2026

MỤC LỤC

TÓM TẮT BÁO CÁO.....	3
1. GIỚI THIỆU CHUNG.....	3
1.1. Bối cảnh và mục tiêu.....	3
1.2. Nhóm thực hiện.....	4
2. QUÁ TRÌNH THỰC TẬP VÀ PHƯƠNG PHÁP LÀM VIỆC.....	4
2.1. Các giai đoạn chính.....	4
2.2. Quản lý cấu hình và V-Model.....	4
2.3. Yêu cầu hệ thống.....	5
2.4. Yêu cầu phần mềm theo thành phần.....	5
2.5. Công cụ và môi trường phát triển.....	6
3. THIẾT KẾ HỆ THỐNG, KIẾN TRÚC VÀ MODULE.....	7
3.1. System design.....	7
3.2. Architecture design.....	8
3.3. Module design.....	9
3.4. Chức năng các tầng.....	10
3.5. Luồng dữ liệu chẩn đoán.....	10
4. PHẦN CỨNG VÀ MẠNG CAN.....	11
4.1. CAN IDs sử dụng.....	12
5. KẾT QUẢ TRIỂN KHAI CÁC THÀNH PHẦN.....	12
5.1. CAN Interface.....	12
5.2. ISO-TP hoàn chỉnh.....	12
5.3. Các dịch vụ UDS.....	13
5.4. DID, dữ liệu động cơ và DTC.....	14
6. GIAO DIỆN VÀ KỊCH BẢN DEMO.....	15
6.1. Thiết kế giao diện.....	15
6.2. Giao diện Read Data by Identifier.....	16
6.3. Kịch bản DTC.....	17
6.4. Minh chứng ISO-TP multi-frame và dữ liệu động.....	18
7. KIỂM THỬ, TRACEABILITY VÀ BẢNG CHỨNG.....	19
7.1. Chiến lược kiểm thử.....	19
7.2. Các kịch bản chính được bao phủ.....	19
7.3. Traceability tiêu biểu.....	19
8. ĐÁNH GIÁ KẾT QUẢ.....	20
8.1. Kết quả đạt được.....	20
8.2. Hạn chế của hiện tại.....	20
8.3. Kiến thức và kỹ năng thu nhận.....	20
9. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	21
TÀI LIỆU VÀ MINH CHỨNG THAM KHẢO.....	21

TÓM TẮT BÁO CÁO

Báo cáo tổng kết quá trình Nhóm 1 thực tập tại FPT Software với đề tài xây dựng mô hình ECU chẩn đoán trên STM32. Hệ thống gồm một node Diagnostic Tester và một node ECU giao tiếp qua CAN, sử dụng ISO-TP để vận chuyển payload và UDS để thực hiện các chức năng chẩn đoán.

Trọng tâm của báo cáo là quá trình hình thành sản phẩm, kiến trúc, các chức năng đạt được, giao diện demo và phương pháp kiểm chứng. Phần mã nguồn chỉ được nhắc ở mức thành phần, không trình bày theo hàm hoặc theo dòng lệnh.

Kết quả: Nhóm đã tích hợp được chuỗi giao tiếp từ giao diện Serial đến CAN, ISO-TP, UDS, DID/DTC và quay video demo trên hai node STM32 thực.

1. GIỚI THIỆU CHUNG

1.1. Bối cảnh và mục tiêu

Trong hệ thống ô tô, CAN là nền tảng truyền thông phổ biến giữa các ECU, còn UDS cung cấp các dịch vụ chẩn đoán. Đề tài được xây dựng như một bộ demonstrator đào tạo: không chỉ lập trình firmware mà còn thực hành quy trình Requirements - Design - Code - Test - Evidence - Release.

- Thiết lập giao tiếp CAN hai chiều giữa node Diagnostic và node ECU trên STM32.
- Tách phần mềm thành các tầng Application, UDS, ISO-TP, CanIf và phần cứng.
- Cung cấp năm dịch vụ UDS theo yêu cầu và các dữ liệu VIN, phần mềm, tốc độ, RPM, nhiệt độ, điện áp.
- Mô phỏng hai DTC và kiểm tra kịch bản không lỗi, một lỗi và nhiều lỗi.
- Tạo giao diện Serial gọn, chỉ hiển thị các chức năng khách hàng cần sử dụng.

1.2. Nhóm thực hiện

STT	Thành viên	Nhóm/đơn vị
1	Nguyễn Đình Cường - 23020786	Nhóm 1 - FPT Software
2	Nguyễn Trường Giang - 23020802	Nhóm 1 - FPT Software
3	Bùi Tiến Dũng - 23020792	Nhóm 1 - FPT Software
4	Bùi Xuân Thanh - 23020883	Nhóm 1 - FPT Software

2. QUÁ TRÌNH THỰC TẬP VÀ PHƯƠNG PHÁP LÀM VIỆC

2.1. Các giai đoạn chính

Giai đoạn	Nội dung trọng tâm	Kết quả
1. Bootcamp	Làm quen CAN, UDS, V-Model, coding rule và GitHub workflow.	Hình thành cách làm việc nhóm và kho mã nguồn.
2. Bring-up CAN	Lắp mạch hai node, cấu hình CAN1, transceiver và CanIf.	Truyền - nhận frame hai chiều trên phần cứng.
3. ISO-TP	Triển khai Single Frame và multi-frame FF/FC/CF.	Vận chuyển được payload UDS dài như VIN.
4. UDS & Data	Tích hợp UDS Dispatcher, DID Manager, DTC Manager và App Engine.	Cung cấp năm dịch vụ UDS và dữ liệu mô phỏng.
5. Integration	Hoàn thiện menu, kiểm thử, log và demo.	Video demo

2.2. Quản lý cấu hình và V-Model

Nhóm áp dụng luồng làm việc GitHub Issue - Branch - Pull Request - Review - Rebase - Merge. Mỗi thay đổi được liên kết với requirement và bằng chứng kiểm thử; nhánh main được xem là nhánh ổn định, không commit trực tiếp.

Kho mã nguồn GitHub của Nhóm 1: https://github.com/xuanthanh-dev/automotive_uds_nhom1

- Requirements xác định dịch vụ, CAN ID, DID, DTC và tiêu chí chấp nhận.
- Thiết kế phân tầng giữ phần UDS không phụ thuộc trực tiếp vào STM32 HAL.
- Unit test kiểm tra module; integration test kiểm tra chuỗi giao tiếp; system/acceptance test tạo bằng chứng trên phần cứng.

2.3. Yêu cầu hệ thống

Tám yêu cầu hệ thống (SYS) được chốt ở đầu dự án và là gốc của toàn bộ chuỗi truy vết yêu cầu - thiết kế - mã nguồn - kiểm thử trình bày ở mục 7.3. Mỗi yêu cầu đi kèm tiêu chí chấp nhận định lượng và phương pháp kiểm chứng.

Mã	Yêu cầu	Tiêu chí chấp nhận	Kiểm chứng
SYS-001	ECU phát bản tin EngineStatus theo chu kỳ	CAN ID 0x100, DLC 8, chu kỳ 1.000 ms $\pm 10\%$, chứa VehicleSpeed, EngineRPM, EngineTemp, BatteryVoltage	ST-001, AT-001
SYS-002	ECU hoạt động như UDS server trên CAN	Hỗ trợ tối thiểu năm SID: 0x10, 0x11, 0x19, 0x22, 0x3E	ST-002..ST-009, AT-002
SYS-003	ECU trả VIN qua DID 0xF190	Phản hồi bắt đầu bằng 62 F1 90, VIN mã ASCII, truyền qua ISO-TP khi dài hơn 7 byte	UT-DID-001, IT-002, ST-002
SYS-004	ECU trả Software Version qua DID 0xF187	Phản hồi 62 F1 87 kèm chuỗi phiên bản phần mềm	UT-DID-001, IT-003, ST-003
SYS-005	ECU cung cấp dữ liệu động qua DID	0x0101 VehicleSpeed, 0x0102 EngineRPM, 0x0103 EngineTemp, 0x0104 BatteryVoltage	UT-DID-001, IT-003, ST-004, ST-005
SYS-006	ECU quản lý và báo cáo DTC	0x010001 quá nhiệt, 0x010002 điện áp thấp; đọc được bằng dịch vụ 0x19	UT-DTC-001/002, IT-004, ST-010, ST-011
SYS-007	ECU trả negative response cho request không hợp lệ	Định dạng 7F <SID> <NRC>; ví dụ 22 FF FF trả về 7F 22 31	UT-UDS-002/003, ST-006
SYS-008	Hệ thống chạy được trên phần cứng thật	STM32 và CAN transceiver hoạt động, có CAN trace hoặc log UART làm bằng chứng	AT-001..AT-004

2.4. Yêu cầu phần mềm theo thành phần

Từ yêu cầu hệ thống, nhóm phân rã thành các yêu cầu phần mềm (SWR) gắn trực tiếp vào từng module. Đây là đầu vào cho bước Architecture design và Module design.

Mã yêu cầu	Nội dung chính	Thành phần
SWR-CAN-001..003	API khởi tạo, API truyền frame (trả về lỗi khi con trả NULL hoặc DLC lớn hơn 8) và callback RxIndication định tuyến theo ID	CanIf

Mã yêu cầu	Nội dung chính	Thành phần
SWR-APP-001/002	Duy trì trạng thái động cơ và đóng gói EngineStatus: byte 0-1 VehicleSpeed, byte 2-3 EngineRPM, byte 4 EngineTemp, byte 5 BatteryVoltage	App Engine
SWR-DID-001/002	DID tĩnh 0xF190, 0xF187 và bốn DID động 0x0101..0x0104	DID Manager
SWR-DTC-001	Set DTC active, clear DTC và trả về danh sách DTC đang active	DTC Manager
SWR-UDS-001..003	Định tuyến theo SID; SID phản hồi tích cực bằng SID request cộng 0x40; negative response theo định dạng 7F <SID> <NRC>	UDS Dispatcher
SWR-ISO-TP-001/002	Single Frame cho payload không quá 7 byte; phân mảnh FF/FC/CF cho payload tới 64 byte	ISO-TP

2.5. Công cụ và môi trường phát triển

Bảng 6. Công cụ và môi trường phát triển

Hạng mục	Lựa chọn của nhóm
Vi điều khiển	STM32F103C8T6 (Cortex-M3), sử dụng ngoại vi CAN1 và USART
Kết nối CAN	PA12 làm CAN_TX, PA11 làm CAN_RX, CAN transceiver ngoài, hai điện trở đầu cuối 120 Ω
Thư viện phần mềm	STM32Cube HAL cho dòng STM32F1, mã ứng dụng viết bằng Embedded C
IDE và cấu hình	STM32CubeIDE với dự án Phase2_FPT, file cấu hình ngoại vi Phase2_FPT.ioc
Nạp và gỡ lỗi	ST-Link qua giao diện SWD
Giao tiếp người dùng	UART tới máy tính, quan sát đồng thời hai cổng COM của tester và ECU
Quản lý mã nguồn	Git và GitHub theo luồng Issue - Branch - Pull Request - Review - Merge
Tài liệu kỹ thuật	docs/requirements, docs/design, docs/tests và docs/process trong kho mã nguồn

3. THIẾT KẾ HỆ THỐNG, KIẾN TRÚC VÀ MODULE

3.1. System design

System design xác định phạm vi demonstrator và các giao diện bên ngoài. Người dùng thao tác trên Diagnostic Tester qua UART; tester trao đổi request/response với ECU qua mạng CAN. ECU sử dụng dữ liệu động cơ mô phỏng và trạng thái fault injection để cung cấp kết quả DID/DTC, trong khi các chức năng kiểm thử nội bộ không xuất hiện trên giao diện khách hàng.



3.2. Architecture design

Architecture design phân rã hai node STM32 thành các tầng có trách nhiệm rõ ràng. Diagnostic Tester gồm giao diện Serial, App Diagnostic, ISO-TP Client và CanIf; ECU gồm Application/Managers, UDS Dispatcher, ISO-TP Server và CanIf. Hai phía chỉ trao đổi CAN frame qua transceiver, nhờ đó logic chẩn đoán không phụ thuộc trực tiếp vào STM32 HAL.



Hình 2. Architecture Design - phân tầng phần mềm node ECU

3.3. Module design

Module design mô tả quan hệ gọi hàm và callback giữa các thành phần triển khai. Ở tester, SerialUi chuyển lựa chọn cho DiagApp, sau đó request đi qua IsoTpClient và CanIf. Ở ECU, frame được chuyển theo chiều ngược lại đến UdsDispatcher; dispatcher truy xuất AppEngine, DidManager hoặc DtcManager trước khi tạo phản hồi.



Hình 3. Module Design - API, trạng thái và mã lỗi từng module

3.4. Chức năng các tầng

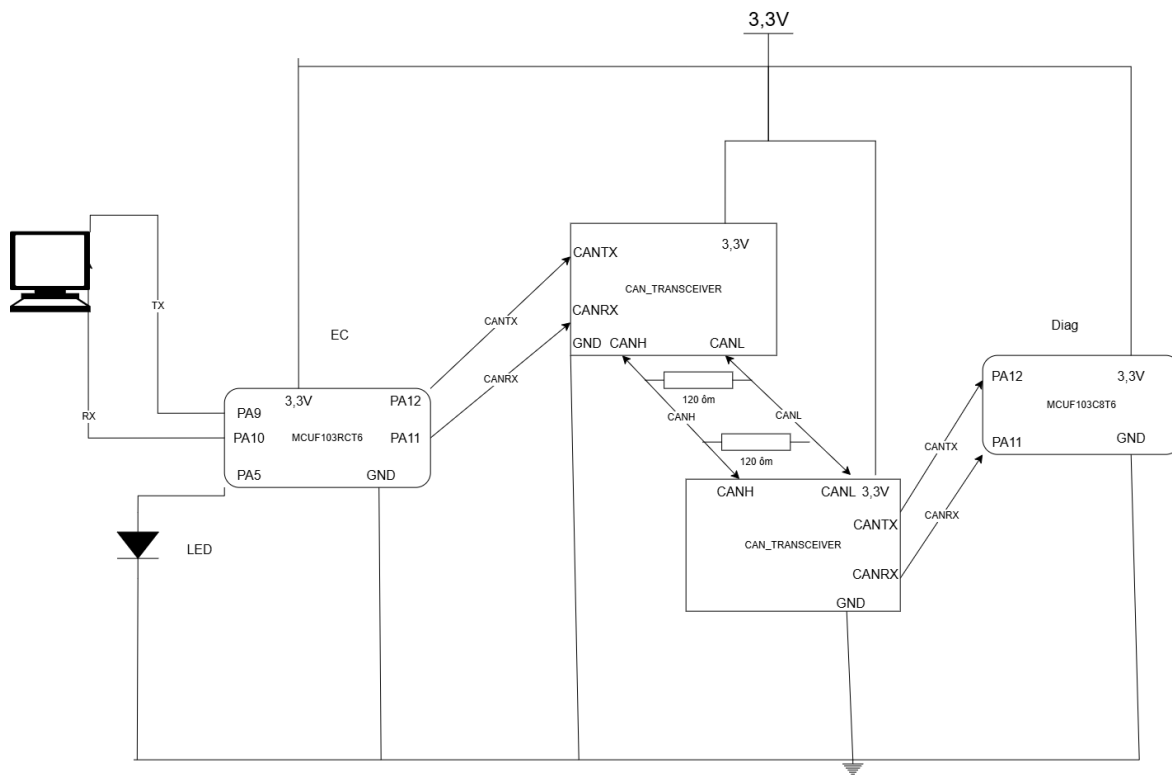
Tầng	Vai trò trong hệ thống
Giao diện / Application	Nhận lựa chọn, quản lý vòng đời request/response và cung cấp dữ liệu động cơ mô phỏng.
UDS	Phân luồng theo SID, xử lý session, reset, DTC, DID, Tester Present và Negative Response
DID / DTC Manager	Quản lý dữ liệu định danh, tín hiệu động và danh sách lỗi đang active.
ISO-TP	Đóng gói payload ngắn trong Single Frame; phân mảnh và ghép payload dài bằng FF/FC/CF.
CanIf	Chuẩn hóa truyền - nhận CAN, kiểm tra frame, routing ID, quản lý FIFO/mailbox và callback.
HAL / Transceiver / Bus	Thực hiện giao tiếp ngoại vi, chuyển đổi tín hiệu logic sang CAN_H/CAN_L và truyền vi sai.

3.5. Luồng dữ liệu chẩn đoán

1. Người dùng chọn chức năng trên menu Serial; App Diagnostic tạo payload UDS.
2. ISO-TP đóng gói payload, CanIf gửi request trên CAN ID 0x7E0.
3. ECU nhận frame, ghép payload ISO-TP và chuyển UDS Dispatcher xử lý.
4. UDS lấy dữ liệu từ App Engine, DID Manager hoặc DTC Manager rồi tạo response.
5. Response được truyền trên ID 0x7E8, ghép lại tại tester và hiển thị dạng dễ đọc.

4. PHẦN CỨNG VÀ MẠNG CAN

Mô hình thử nghiệm sử dụng hai STM32F103C8T6, mỗi board kết nối một CAN transceiver qua PA12 (CAN_TX) và PA11 (CAN_RX). CAN_H nối CAN_H, CAN_L nối CAN_L, hai node dùng chung GND và hai đầu bus có điện trở 120 Ω .



Hình 4. Sơ đồ kết nối phần cứng hai node STM32

4.1. CAN IDs sử dụng

CAN ID	Chiều truyền	Mục đích
0x7E0	Diagnostic -> ECU	UDS request
0x7E8	ECU -> Diagnostic	UDS response

5. KẾT QUẢ TRIỂN KHAI CÁC THÀNH PHẦN

5.1. CAN Interface

CanIf là lớp nền tảng được xây dựng sớm để phần mềm tầng trên không thao tác trực tiếp với STM32 HAL. Kết quả chính gồm khởi tạo CAN, truyền frame chuẩn 11 bit, nhận theo FIFO/ngắt, kiểm tra DLC và route các ID sử dụng trong demonstrator.

5.2. ISO-TP hoàn chỉnh

Tầng ISO-TP hỗ trợ payload tối đa 64 byte. Payload không quá 7 byte được gửi bằng Single Frame; payload dài hơn được truyền bằng First Frame, Flow Control và các Consecutive Frame. Module quản lý sequence number, Block Size, STmin, trạng thái TX/RX và timeout N_Bs/N_Cr 1.000 ms.

Byte đầu tiên của mỗi frame là PCI (Protocol Control Information): bốn bit cao xác định loại frame, bốn bit thấp mang giá trị phụ thuộc loại frame.

Loại frame	Byte 0 (PCI)	Các byte tiếp theo	Ghi chú
Single Frame (SF)	0x0L	L byte payload	L từ 1 đến 7
First Frame (FF)	0x1H	0xLL và 6 byte payload đầu	12 bit độ dài, tối đa 64 byte
Flow Control (FC)	0x30 + FS	Block Size, STmin	FS: 0 = Continue To Send, 1 = Wait, 2 = Overflow
Consecutive Frame (CF)	0x2N	Tối đa 7 byte payload	N là sequence number 1..15 rồi quay vòng về 0

Module vận hành hai máy trạng thái độc lập: phía truyền đi qua IDLE - WAIT_FC - SENDING_CF, phía nhận đi qua IDLE - RECEIVING_CF, kèm hai bộ định thời N_Bs và N_Cr đặt 1.000 ms. Mọi tình huống bất thường được ghi lại bằng mã lỗi riêng cùng bộ đếm lỗi - gồm con trỏ NULL, độ dài bằng 0 hoặc vượt buffer, byte PCI không hợp lệ, sai sequence number, nhận CF hoặc FC không đúng trạng thái, Flow Control báo overflow, timeout truyền và timeout nhận - nhờ đó khi kiểm thử có thể xác định chính xác nguyên nhân thất bại.

5.3. Các dịch vụ UDS

SID	Dịch vụ	Kết quả mong đợi	Positive Response
0x10	Diagnostic Session Control	Default / Extended Session	0x50
0x11	ECU Reset	Soft reset sau khi gửi phản hồi	0x51
0x19	Read DTC Information	Trả danh sách DTC active	0x59
0x22	Read Data by Identifier	Trả VIN, SW và dữ liệu động	0x62
0x3E	Tester Present	Duy trì phiên chẩn đoán	0x7E

Khi request không hợp lệ, UDS Dispatcher không im lặng mà trả về phản hồi phủ định theo định dạng 7F <SID gốc> <NRC>. Dự án hỗ trợ sáu NRC theo yêu cầu SWR-UDS-003.

NRC	Tên theo ISO 14229-1	Trường hợp phát sinh
0x11	serviceNotSupported	SID không nằm trong năm dịch vụ được hỗ trợ; ví dụ request 99 trả về 7F 99 11
0x12	subFunctionNotSupported	Sub-function không hợp lệ, ví dụ mã session hoặc reset type không được hỗ trợ
0x13	incorrectMessageLengthOrInvalidFormat	Độ dài request không đúng với dịch vụ được yêu cầu
0x22	conditionsNotCorrect	Điều kiện thực thi chưa thỏa mãn tại thời điểm nhận request
0x31	requestOutOfRange	DID không tồn tại; ví dụ request 22 FF FF trả về 7F 22 31
0x33	securityAccessDenied	Chưa được cấp quyền truy cập; dự phòng cho phần mở rộng, chưa dùng trong demo

5.4. DID, dữ liệu động cơ và DTC

DID / Nguồn	Dữ liệu	Ghi chú
EngineStatus / 0x100	Tốc độ, RPM, nhiệt độ, điện áp	Bản tin chu kỳ được tester lưu và hiển thị
0xF190	VIN 17 byte ASCII	Cần ISO-TP multi-frame
0xF187	Software Version 8 byte	Có thể cần multi-frame
0x0101	Vehicle Speed	Dữ liệu mô phỏng
0x0102	Engine RPM	Dữ liệu mô phỏng
0x0103	Engine Temperature	Kích hoạt over-temperature khi vượt ngưỡng
0x0104	Battery Voltage	Kích hoạt low-battery khi thấp hơn ngưỡng

6. GIAO DIỆN VÀ KỊCH BẢN DEMO

6.1. Thiết kế giao diện

Theo yêu cầu, Menu chính chỉ gồm năm dịch vụ chẩn đoán và lựa chọn thoát.



Hình 5. Giao diện chính trên Diagnostic Tester

Phím	Chức năng
1	Diagnostic Session Control
2	ECU Reset
3	Read DTC Information
4	Read Data by Identifier
5	Tester Present
6	Quit

6.2. Giao diện Read Data by Identifier

Khi chọn option 4, tester chuyển sang menu con. Read EngineStatus hiển thị dữ liệu chu kỳ đã nhận; các mục còn lại tạo request UDS 0x22 theo DID tương ứng. Sau mỗi thao tác, giao diện trở lại menu con để người dùng tiếp tục truy vấn.



Hình 6. Menu Read Data by Identifier và kết quả đọc dữ liệu

Phím	Lựa chọn	Nguồn dữ liệu
1	Read EngineStatus	CAN ID 0x100
2	Read VIN	DID 0xF190
3	Read SW Info	DID 0xF187
4	Read Vehicle Speed	DID 0x0101
5	Read Engine RPM	DID 0x0102
6	Read Engine Temp	DID 0x0103
7	Read Battery Voltage	DID 0x0104
8	Back	Trở về menu chính

6.3. Kịch bản DTC

Fault injection được thực hiện tại phía ECU hoặc trong thiết lập kiểm thử, không được đưa thành một option trên giao diện khách hàng. Khi người dùng chọn Read DTC Information, tester chỉ hiển thị kết quả DTC hiện tại.

Trạng thái	DTC active	Kết quả hiển thị
Mới khởi động	Không có	0 lỗi / No DTC detected
Inject quá nhiệt	0x010001	Engine Over Temperature
Inject điện áp thấp	0x010002	Low Battery
Inject cả hai lỗi	0x010001 + 0x010002	Trả về đồng thời hai DTC

```

===== DIAGNOSTIC MENU =====
1. Diagnostic Session Control
2. ECU Reset
3. Read DTC Information
4. Read Data by Identifier
5. Tester Present
6. Quit
Select:
Received: 3
Received HEX: 0x33

[DEBUG] diagApp.state = 1

[DIAG] TX: 19 02 09
[DIAG] RX: 59 02 09
No DTC detected.

===== DIAGNOSTIC MENU =====
1. Diagnostic Session Control
2. ECU Reset
3. Read DTC Information
4. Read Data by Identifier
5. Tester Present
6. Quit

```

Hình 7. Trạng thái ban đầu: không có DTC active

6.4. Minh chứng ISO-TP multi-frame và dữ liệu động

Khung hình demo bên dưới ghép hai vùng log: bên trái là kết quả hướng người dùng, bên phải là log ECU nội bộ. Log cho thấy request VIN 22 F1 90, UDS response 62 F1 90..., First Frame 10 14... và Flow Control 30 00..., đồng thời minh họa việc đọc Engine Temperature.

4. Read Vehicle Speed	[EC] CAN TX 0x7E8: 05 62 01 01 00 4E 00 00
5. Read Engine RPM	[EC] CAN RX 0x7E0: 03 22 F1 90 00 00 00 00
6. Read Engine Temp	[EC] UDS RX: 22 F1 90
7. Read Battery Voltage	[EC] UDS TX: 62 F1 90 56 4E 55 55 45 54 32 30 32 36
8. Back	53 54 4D 33 32 30 31
Select:	[EC] CAN TX 0x7E8: 10 14 62 F1 90 56 4E 55
Received: 6	[EC] CAN RX 0x7E0: 30 00 00 00 00 00 00 00
Received HEX: 0x36	[EC] CAN RX 0x7E0: 03 22 F1 87 00 00 00 00
	[EC] UDS RX: 22 F1 87
[DEBUG] diagApp.state = 1	[EC] UDS TX: 62 F1 87 56 31 2E 30 2E 30 30 31
	[EC] CAN TX 0x7E8: 10 0B 62 F1 87 56 31 2E
[DIAG] TX: 22 01 03	[EC] CAN RX 0x7E0: 30 00 00 00 00 00 00 00
[DIAG] RX: 62 01 03 52	[EC] CAN RX 0x7E0: 03 22 01 01 00 00 00 00
Engine Temp: 82 C	[EC] UDS RX: 22 01 01
	[EC] UDS TX: 62 01 01 00 48
===== READ DATA BY IDENTIFIER =====	[EC] CAN TX 0x7E8: 05 62 01 01 00 48 00 00
1. Read EngineStatus	[EC] CAN RX 0x7E0: 03 22 01 02 00 00 00 00
2. Read VIN	[EC] UDS RX: 22 01 02
3. Read SW Info	[EC] UDS TX: 62 01 02 08 8E
4. Read Vehicle Speed	[EC] CAN TX 0x7E8: 05 62 01 02 08 8E 00 00
5. Read Engine RPM	[EC] CAN RX 0x7E0: 03 22 01 03 00 00 00 00
6. Read Engine Temp	[EC] UDS RX: 22 01 03
7. Read Battery Voltage	[EC] UDS TX: 62 01 03 52
8. Back	[EC] CAN TX 0x7E8: 04 62 01 03 52 00 00 00

Hình 8. Demo DID và ISO-TP multi-frame giữa tester - ECU

Video demo: [Tại đây](#)

7. KIỂM THỬ, TRACEABILITY VÀ BẰNG CHỨNG

7.1. Chiến lược kiểm thử

Kiểm thử được tổ chức theo các mức của V-Model. Giao diện khách hàng không hiển thị tên test; ID test, log chi tiết và traceability được giữ trong tài liệu nội bộ.

Mức kiểm thử	Phạm vi	Bằng chứng trong project
Unit Test	CanIf, DID, DTC, UDS, ISO-TP	Test specification và các test case theo module
Integration Test	App -> CanIf; CAN -> ISO-TP -> UDS; UDS -> DID/DTC	Chuỗi IT-001 đến IT-006
System Test	EngineStatus và các dịch vụ UDS	ST-001 đến ST-011 trong tài liệu nội bộ
Acceptance Test	Phần cứng, luồng UDS, fault injection, GitHub process	AT-001 đến AT-004 và video demo

7.2. Các kịch bản chính được bao phủ

- Truyền CanIf hợp lệ, từ chối con trỏ NULL và DLC lớn hơn 8.
- ISO-TP Single Frame; chuỗi First Frame - Flow Control - Consecutive Frame; sai sequence và timeout.
- Read VIN, SW version, bốn DID động và DID không hợp lệ.
- Default/Extended Session, ECU Reset, Tester Present và negative response.
- DTC ban đầu bằng 0; quá nhiệt; điện áp thấp; cả hai lỗi cùng active.
- End-to-end trên hai STM32 với CAN ID 0x7E0/0x7E8 và log UART hai chiều.

7.3. Traceability tiêu biểu

Yêu cầu	Thành phần	Kiểm chứng
SYS-002	UDS + ISO-TP + CanIf	IT-002, ST-002..ST-009, AT-002
SYS-003	DID VIN 0xF190	UT-DID-001, IT-002, ST-002
SYS-006	DTC Manager + UDS 0x19	UT-DTC-001/002, IT-004, ST-010/011
SWR-ISO-TP-001/002	Single Frame + multi-frame	Unit/integration log và demo VIN
SYS-008	Hai STM32 + CAN transceiver	Acceptance test và video demo

8. ĐÁNH GIÁ KẾT QUẢ

8.1. Kết quả đạt được

- Hoàn thành mô hình giao tiếp CAN hai node trên phần cứng STM32.
- Xây dựng được kiến trúc phân tầng và giữ ranh giới trách nhiệm tương đối rõ ràng.
- Tích hợp ISO-TP đủ SF, FF, FC, CF cho payload UDS ngắn và dài.
- Cung cấp năm dịch vụ UDS, sáu nguồn dữ liệu chẩn đoán và hai DTC mô phỏng.
- Thiết kế lại menu theo hướng khách hàng và quay video demo có log tester/ECU.
- Thực hành quy trình Issue, Branch, Pull Request, Review, Test Evidence và Traceability.

8.2. Hạn chế của hiện tại

- ISO-TP được xây dựng cho demonstrator, chưa tuyên bố tuân thủ timing sản phẩm.
- DTC lưu trong RAM và mất khi reset; chưa có NVM hay cơ chế ageing/healing.
- Mô hình chưa kết nối với xe thực và không thay thế ECU sản phẩm.
- Bảng chứng demo nên được đóng gói thêm bằng CAN trace và bảng pass/fail cuối cùng trước release.

8.3. Kiến thức và kỹ năng thu nhận

Qua đề tài, nhóm hiểu rõ hơn sự khác nhau giữa CAN controller, CAN transceiver và giao thức tầng trên; cách ISO-TP giải quyết giới hạn 8 byte của CAN Classic; cách UDS phân tách service với DID/DTC; và tầm quan trọng của log, test evidence, code review, traceability trong dự án embedded.

9. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Quá trình thực tập đã giúp Nhóm 1 đi từ kiến thức bootcamp đến một demonstrator chẩn đoán có thể hoạt động trên phần cứng thực. Giá trị lớn nhất không nằm ở số lượng dòng code mà ở việc nhóm đã thực hành một chuỗi kỹ thuật hoàn chỉnh: phân tích yêu cầu, thiết kế phân tầng, tích hợp phần cứng, kiểm thử và trình diễn kết quả theo góc nhìn khách hàng.

Trong bước hoàn thiện release, nhóm cần đồng bộ giao diện mới vào nhánh chính, chạy lại toàn bộ test, gắn bằng chứng pass/fail vào traceability matrix, đóng tag phiên bản và lưu trữ video/CAN trace cùng tài liệu release.

Kết luận: Mục tiêu xây dựng nền tảng CAN - ISO-TP - UDS, giao diện chẩn đoán và kịch bản DTC trên hai STM32 đã được thể hiện trong snapshot và video demo; công việc còn lại chủ yếu là đồng bộ, đóng gói và phát hành bản cuối.

TÀI LIỆU VÀ MINH CHỨNG THAM KHẢO

- Kho mã nguồn GitHub của Nhóm 1: https://github.com/xuanthanh-dev/automoiive_uds_nhom1